

Caso de Estudio: Gestión de Líneas de Transmisión del Sistema Eléctrico Nacional (POO)

1. Objetivo general

Desarrollar una aplicación orientada a objetos en Java (NetBeans) que permita gestionar las líneas de transmisión eléctrica de las regiones: Cauca-Nariño, Santander, Boyacá-Casanare, Subárea CQR, Bogotá, Antioquia, Valle, GCM, Huila-Tolima, Cordoba-Sucre aplicando los principios de diseño POO y la construcción de una interfaz gráfica con operaciones CRUD-S (Crear, Leer, Actualizar, Eliminar y Buscar).

2. Objetivos específicos

1. Diseñar el modelo UML que represente las entidades principales del dominio eléctrico: línea de transmisión, subestación, conexión, capacidad y ubicación.
2. Implementar un proyecto NetBeans con GUI (Swing o JavaFX) que permita gestionar estas entidades.
3. Incorporar mecanismos para abrir y guardar información (archivos CSV o JSON).
4. Aplicar principios de abstracción, encapsulamiento y relaciones (agregación y composición) entre las clases.
5. (Opcional – Puntos adicionales) Incluir un componente de mapa que visualice las líneas geográficamente.

3. Contexto

El sistema eléctrico de la Costa Caribe conecta subestaciones y líneas de transmisión que transportan energía entre los departamentos de Atlántico, Bolívar, Córdoba, Sucre, Cesar, Magdalena y La Guajira. Cada línea tiene atributos técnicos como voltaje, corriente nominal, capacidad y longitud.

4. Actividades (por fases)

Fase 1 – Análisis del dominio y modelo UML

Identificar las clases principales del dominio eléctrico. Ejemplo:

- LineaTransmision: id, nombre, voltajeNominal, corrienteNominal, capacidadMW, longitudKm.
- Subestacion: id, nombre, municipio, departamento, coordenadas (latitud, longitud).
- SistemaElectrico: gestiona la colección de líneas y subestaciones.

Determinar relaciones: una línea conecta dos subestaciones (composición o agregación).

Entregable 1: archivo UML (.png o .uml) generado con PlantUML, Visual Paradigm, StarUML o similar.

Fase 2 – Implementación en NetBeans (Java)

Crear un proyecto con los siguientes paquetes y clases:

1. Paquete modelo: clases con atributos privados, constructores, getters y setters, y métodos de negocio.

Ejemplo de método:

```
public double calcularCapacidadMW(double voltajeKV, double corrienteA, double pf);
```

2. Paquete persistencia: clases para lectura/escritura de archivos CSV o JSON.

Ejemplo de métodos:

```
public void guardarLineas(List<LineaTransmision> lineas, String ruta);
```

```
public List<LineaTransmision> cargarLineas(String ruta);
```

3. Paquete vista (GUI): interfaz gráfica con menús y botones CRUD-S.

4. Paquete controlador: coordina las acciones entre vista y modelo.

Fase 3 – Funcionalidades requeridas

- CRUD-S completo: crear, leer, actualizar, eliminar y buscar líneas.

- Gestión de archivos: abrir y guardar información en CSV o JSON.

- Cálculo automático de capacidad (MW): $MW = \sqrt{3} \times V_{kV} \times I_A / 1000 \times pf$ (usar $pf = 0.95$ por defecto).

- Validación de datos: no permitir campos vacíos o negativos.

Fase 4 – Extensión (Puntos adicionales)

1. Incluir una visualización geográfica usando JXMapView (Swing) o JavaFX WebView (OpenStreetMap/Google Maps).

2. Permitir filtrar líneas por voltaje o departamento.

3. Implementar una función para simular la carga o capacidad total del sistema.

5. Entregables

Entregable	Descripción	Formato
1. Modelo UML	Diagrama de clases con relaciones, atributos y métodos.	.png o .uml
2. Proyecto NetBeans	Aplicación funcional con interfaz gráfica y CRUD-S.	.zip
3. Informe breve (opcional)	Explicación del diseño y capturas de pantalla del programa.	.pdf
4. Archivos fuentes	Datos técnicos de cada línea de transmisión (paractec.xm.com.co)	.xlsx

6. Evaluación sugerida

Criterio	Descripción	Ponderación
Diseño orientado a objetos	Uso de encapsulamiento, relaciones y buen diseño UML	25%
Implementación en Java	Correcta programación y estructura del proyecto	25%
Interfaz gráfica	Cumple CRUD-S y es funcional	20%
Persistencia	Guarda y abre archivos correctamente	20%
Extensión (mapa o visualización)	Funcionalidad adicional y creatividad	10%